



在线SQL判题考试系统

数据库设计报告

汇报人：孙晓博 | 小组成员：黄竺、刘翔宇 | 日期：2026年5月

目录CONTENTS

01

系统概述

项目背景与目标

02

系统需求分析

实体与属性定义 | 实体间关系分析 | 核心业务流程与功能

03

数据库概念模型设计

ER模型概述与实体关系图 | 完整性约束分析

04

数据库逻辑与物理设计

关系模式转换与范式 | 完整性约束实现

05

功能模块划分

各模块与数据表映射

06

总结与展望

设计总结与未来优化方向

01 系统概述

在线SQL判题与考试系统

ONLINE SQL JUDGING & EXAMINATION SYSTEM

核心目标：为数据库课程教学提供一个高效、自动化的在线实践与评估平台，连接“教”与“学”。



学生端 · 学习与评估

- 在线进行 SQL 语句练习与提交
- 参加由教师组织的限时在线考试
- 实时查看判题结果与历史记录，查漏补缺



教师端 · 教学与管理

- 管理题库：创建、编辑题目及测试用例
- 组织考试：灵活设置时间、题目与分值
- 教学评估：查看提交记录与成绩统计



管理员/助教端 · 维护与辅助

- 用户全生命周期管理与角色分配
- 系统日常维护与功能权限配置
- 协助教师进行考试监督与答疑辅助



设计价值：效率与反馈的双重升级

通过自动化判题与成绩统计，显著减轻教师的重复工作，提升教学效率；同时为学生提供即时、客观的反馈，缩短“练习-反馈”周期，切实增强学习效果。

02 需求分析 - 实体及属性定义

SYSTEM CORE ENTITIES & ATTRIBUTES / 系统核心实体及其属性定义

User | 用户实体

属性: user_id, username, password_hash, role, created_at
约束: user_id(自增PK), username(唯一-非空), role(枚举:学生/教师/管理员/助教)

Class | 班级实体

属性: class_id, class_name, teacher_id
约束: class_id(主键), teacher_id(外键 -> User表)

StudentClass | 学生-班级关联

属性: student_id, class_id
约束: (student_id, class_id)复合主键, 分别关联User与Class表

Question | 题目实体

属性: question_id, title, description, answer_sql, creator_id
约束: question_id(主键), creator_id(外键 -> User表)

TestCase | 测试用例实体

属性: case_id, question_id, input_sql, expected_output
约束: case_id(主键), question_id(外键 -> Question表)

Exam | 考试实体

属性: exam_id, exam_name, start_time, end_time, creator_id
约束: exam_id(主键), creator_id(外键 -> User表)

ExamQuestion | 试卷题目关联

属性: exam_id, question_id, score
约束: (exam_id, question_id)复合主键, 分别关联Exam与Question表

ExamStudent | 考试学生关联

属性: exam_id, student_id, final_score (DEFAULT 0)
约束: (exam_id, student_id)复合主键, 分别关联Exam与User表

Submission | 提交记录实体

属性: submission_id, user_id, question_id, exam_id, status, score, submit_time
约束: submission_id(主键), status(枚举:AC/WA/ERROR)

02 需求分析 - 实体间联系

实体间关系分析 (Relationship Analysis)

1 : n

User (教师) – Class
一位教师可以创建和管理多个班级。

n : m

User (学生) – Class
一个学生可以加入多个班级，一个班级有多个学生。通过 StudentClass 关联表实现。

1 : n

User (教师/管理员) – Question
一位用户（教师/管理员）可以创建多道题目。

1 : n

Question – TestCase
一道题目需要包含多个测试用例来验证SQL的正确性。

1 : n

User (教师) – Exam
一位教师可以创建多场考试。

n : m

Exam – Question
一场考试包含多道题目，一道题目可出现在多场考试中。通过 ExamQuestion 关联并记录分值。

n : m

Exam – User (学生)
一场考试有多名学生参加，一名学生可参加多场考试。通过 ExamStudent 记录最终成绩。

1 : n

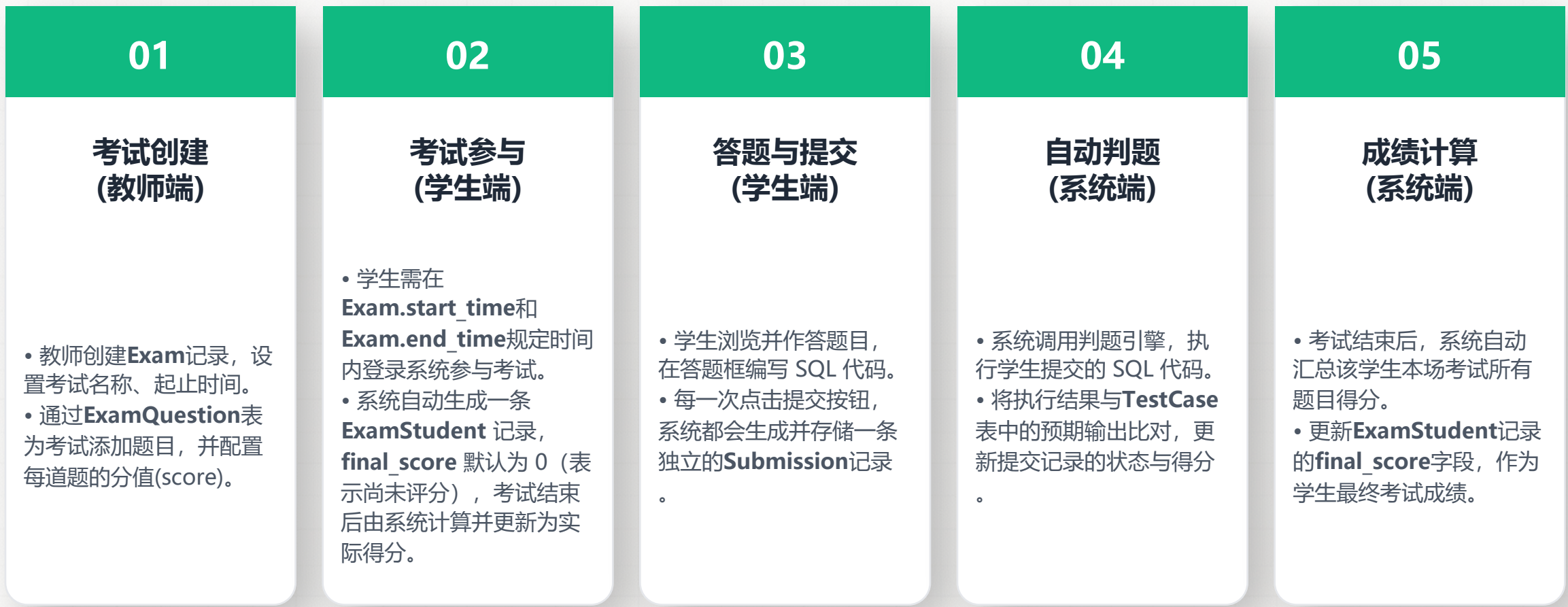
User / Question / Exam – Submission
用户、题目、考试均可对应多次提交记录，实现完整的提交记录追踪。



设计说明：关系定义清晰反映了现实业务逻辑。所有多对多关系均通过引入关联实体正确建模，避免数据冗余，保证数据一致性。

02 需求分析 - 核心业务流程

核心业务流程：学生参加在线考试



02 需求分析 - 角色功能矩阵

各角色功能分配与权限界定

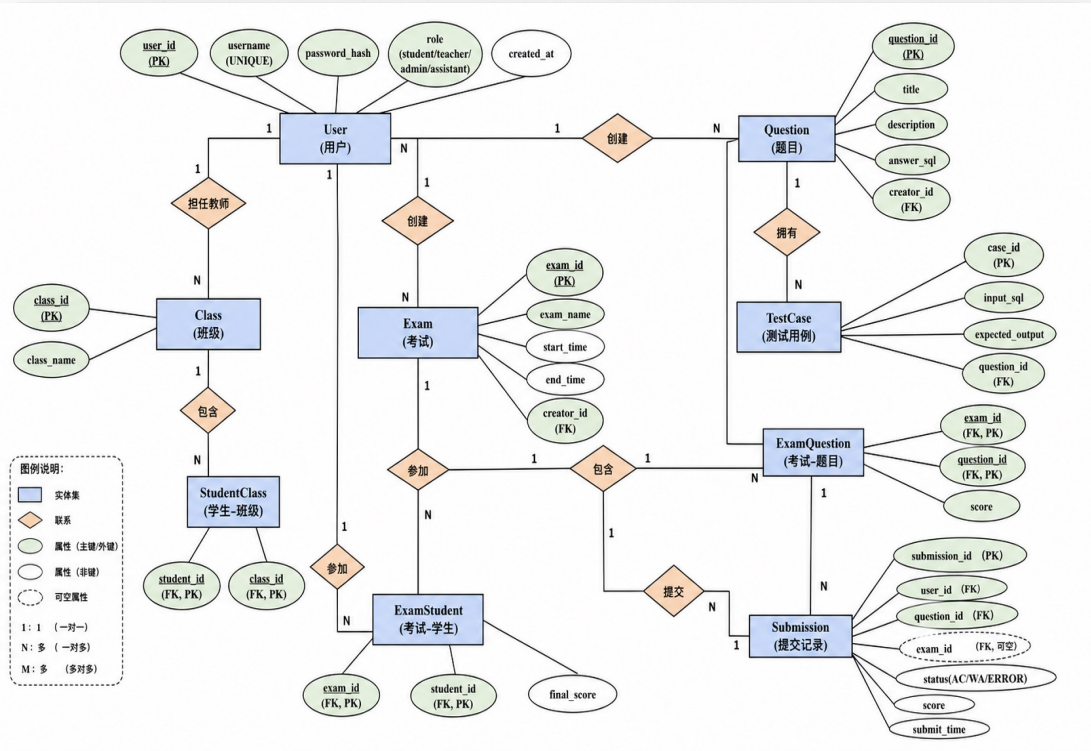
学生 (Student)	教师 (Teacher)	管理员 (Admin)	助教 (Assistant)
👤 用户管理：注册 / 登录账号 🏠 班级管理：查看班级 / 申请加入 📁 题库管理：浏览题库 / 练习题目 📝 考试管理：参加考试 / 查看试卷 ✅ 判题提交：提交SQL / 实时查看结果 📊 统计查询：查看个人历史成绩记录	👤 用户管理：查看及编辑个人信息 🏠 班级管理：创建 / 解散 / 管理班级 📁 题库管理：增 / 删 / 改题目及测试用例 📝 考试管理：创建考试 / 发布及回收 ✅ 判题提交：查看所有学生提交记录 📊 统计查询：查看班级整体成绩统计	👤 用户管理： • 系统级用户增/删/改 • 为用户分配不同角色 • 重置用户密码 🏠 其他管理： • 负责系统全局配置与维护 • 数据备份与权限审计	👤 用户管理：仅可查看个人信息 🏠 班级管理：查看被授权的指定班级 📁 题库管理：无操作权限 📝 考试管理：无操作权限 ✅ 判题提交：辅助查看班级提交记录 📊 统计查询：查看班级成绩统计报表

💡设计原则：遵循“最小权限原则”。教师聚焦内容生产，学生聚焦学习实践，管理员保障系统稳定，助教提供教学辅助。

03 概念模型 - ER图

实体关系图 (Entity Relationship Diagram)

本设计采用陈氏表示法，直观清晰地展示了系统中的9个关键实体以及它们之间的逻辑关联。

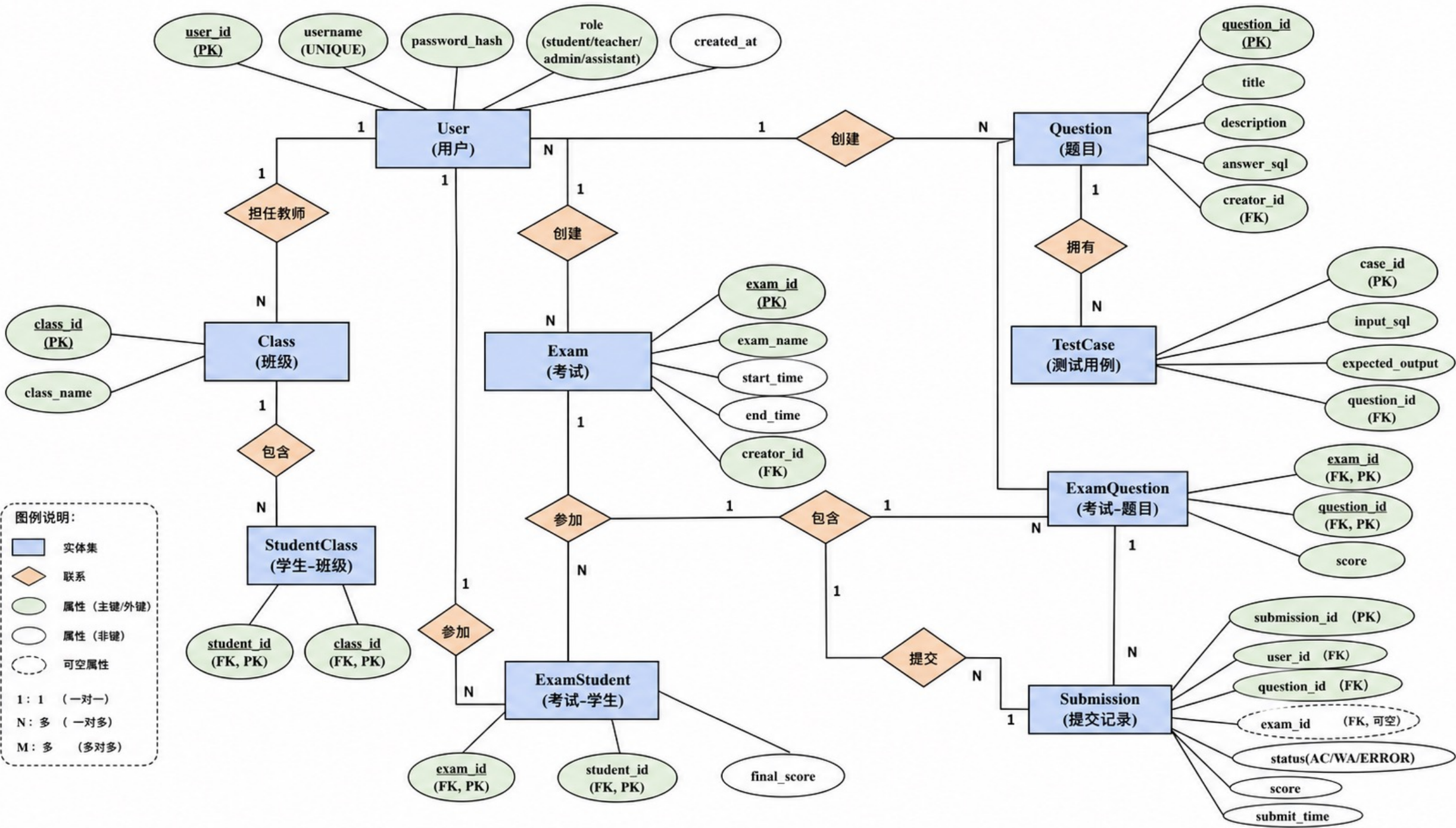


核心实体概览

User (用户):系统核心，所有操作的执行者与权限主体。
Class (班级) & Question (题目):教学的基本组织单元与核心教学资源。
Exam (考试) & Submission (提交):主要的教学评估手段与学生的实践行为记录。



关联实体解释: StudentClass, ExamQuestion, ExamStudent 作为关联实体，有效解决了实体间复杂的 “多对多” 关系问题。



03 概念模型 - 完整性约束

实体属性的完整性约束定义

UserPK: user_id | UQ: username | NN: username, password_hash, role | DF: created_at (CURRENT_TIMESTAMP)

ClassPK: class_id | FK: teacher_id → User | NN: class_name, teacher_id

StudentClassPK: (student_id, class_id) | FK: student_id→User, class_id→Class | NN: 所有字段

QuestionPK: question_id | FK: creator_id → User

TestCasePK: case_id | FK: question_id → Question | NN: question_id

ExamPK: exam_id | FK: creator_id → User | NN: exam_name, start_time, end_time

ExamQuestionPK: (exam_id, question_id) | FK: exam_id→Exam, question_id→Question | NN: score

ExamStudent:PK: (exam_id, student_id) | FK: exam_id→Exam, student_id→User | NN: final_score
Submission:PK: submission_id | FK: user_id, question_id, exam_id | NN: status | DF: submit_time

CHECK约束: Exam起止时间; 所有分值0-100。

03 概念模型 - 关系与键的选择

关系种类总结

一对多 (1:n)

- User (教师) → Class | User → Question | Question → TestCase
- User (教师) → Exam | User → Submission | Question → Submission
- Exam → Submission

多对多 (n:m)

- User (学生) ↔ Class (通过 StudentClass)
- Exam ↔ Question (通过 ExamQuestion)
- Exam ↔ User (学生) (通过 ExamStudent)

键的选择分析

主键 (Primary Key)

- **独立实体**: 采用自增整数 ID (如 User, Question), 简单高效, 适合大多数场景。
- **关联实体**: 采用复合主键 (如 StudentClass), 直接反映关联本质, 无需额外的无意义ID。

外键 (Foreign Key)

- 所有外键均严格且准确地指向了对应父表的主键, 确保数据引用完整性, 建立清晰的逻辑关系网络。

04 数据库设计 - 关系模式转换

从 ER 模型到关系模式的转换：标准化与策略

设计原则：第三范式 (3NF)

定义： 满足第二范式(2NF)，且所有非主属性都不传递依赖于主键，即非主属性必须直接依赖于主键。

本设计实现：

- 彻底消除了部分函数依赖与传递依赖，保证数据结构清晰。
- 示例：`User` 表中 `role` 直接依赖于 `user\_id`，无冗余中间依赖。
- 示例：`Class` 表通过 `teacher\_id` 直接关联 `User`，逻辑直接，规避更新异常。

核心表设计策略

映射规则： 遵循“实体-表”——对应原则，实体属性直接转为表字段。

多对多关系处理： 引入独立关联表（如 `StudentClass`，`ExamQuestion`）作为桥梁，将多对多拆解为两个一对多关系，这是行业标准且高效的做法。

键约束与索引优化：

- **主键：** 采用 `INT AUTO\_INCREMENT` 或业务复合主键，确保唯一性。
- **外键：** 严格定义外键约束以维护数据完整性，建议为外键列创建索引以优化查询性能。

补充：Submission.score 非空约束。

04 数据库设计 - 完整性约束实现

从实体、参照到业务规则的三层防护网构建



实体完整性

实现方式：`PRIMARY KEY` 主键约束、
`AUTO\_INCREMENT` 自增策略、
`UNIQUE` 唯一索引。

✓ **完全实现 · 评价：**设计严密，确保所有数据表的记录具有唯一标识，杜绝重复或无主数据。



参照完整性

实现方式：所有外键均采用 ON DELETE RESTRICT ON UPDATE RESTRICT。

✓ **完全实现 · 评价：**保证了表间关联的有效性，彻底避免了“悬空引用”和数据孤岛，维护了数据整体的逻辑一致性。



用户定义完整性 (业务规则)

实现方式：已在数据库层面利用 `CHECK` 约束实现了核心业务规则校验（如考试起止时间、题目分值、最终成绩等），形成了“应用层 + 数据库层”双重防线。

✓ **完全实现 · 评价：**核心业务规则均已下沉至数据库层，与应用层校验构成双重防线，彻底避免了业务语义违规数据入库，数据完整性达到最高防护等级。



设计总结：数据库的核心骨架（实体与参照完整性）已稳固建立，足以支撑系统运行。建议在下一阶段，将部分高频业务规则通过 `CHECK` 约束在数据库层落地，以减少应用层逻辑压力并防止数据绕过业务层被非法修改，从而构建更坚固的数据防线。

05 功能模块划分

系统功能模块及其对应的数据表映射关系



用户管理模块

核心表: User

操作: INSERT(注册), SELECT(登录),
UPDATE(修改), DELETE



班级管理模块

核心表: Class, StudentClass

操作: 班级增删改查、学生批量加入/退出
班级关联



题库管理模块

核心表: Question, TestCase

操作: 对题目内容和测试用例进行完整的
CRUD 操作



考试管理模块

核心表: Exam, ExamQuestion,
ExamStudent

操作: 考试全生命周期管理, 关联题目, 记
录成绩



判题与提交模块

核心表: Submission

操作: INSERT 提交记录, UPDATE 实时判
题状态与分数



统计查询模块

核心表: 关联查询所有数据表

操作: 复杂 JOIN 查询, 生成排名、提交记
录分析等报表

06 总结与展望



完整性与一致性

通过合理的主键、外键和约束设计，确保了数据的实体完整性和参照完整性，从源头保证数据质量。



结构清晰与规范化

严格遵循第三范式(3NF)，有效消除了数据冗余和更新异常，保证了数据库结构的长期稳定和可维护性。



功能完备性

基于9张核心表，完整覆盖了从题库管理、试卷组卷、考试执行到在线判题的所有核心业务流程。



可扩展性

当前设计为未来的功能扩展预留了充足空间，例如可轻松增加题目标签体系、多维度考试分类及详细判题日志等功能。



结论：本设计是一个成熟、稳健且可直接用于开发的数据库方案，能够全面满足在线SQL判题考试系统的核心业务需求。

```
CREATE DATABASE IF NOT EXISTS oj_system_course
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE oj_system_course;

-- 1. 用户表
CREATE TABLE User (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('student','teacher','admin','assistant') NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 2. 班级表
CREATE TABLE Class (
    class_id INT AUTO_INCREMENT PRIMARY KEY,
    class_name VARCHAR(50) NOT NULL,
    teacher_id INT NOT NULL,
    FOREIGN KEY (teacher_id) REFERENCES User(user_id),
    INDEX (teacher_id)
);

-- 3. 学生-班级关联表
CREATE TABLE StudentClass (
    student_id INT,
    class_id INT,
    PRIMARY KEY (student_id, class_id),
    FOREIGN KEY (student_id) REFERENCES User(user_id),
    FOREIGN KEY (class_id) REFERENCES Class(class_id),
    INDEX (class_id)
);
```


-- 4. 题目表

```
CREATE TABLE Question (  
    question_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(100) NOT NULL,  
    description TEXT,  
    answer_sql TEXT,  
    creator_id INT,  
    FOREIGN KEY (creator_id) REFERENCES User(user_id),  
    INDEX (creator_id)  
);
```

-- 5. 测试用例表

```
CREATE TABLE TestCase (  
    case_id INT AUTO_INCREMENT PRIMARY KEY,  
    question_id INT,  
    input_sql TEXT,  
    expected_output TEXT,  
    FOREIGN KEY (question_id) REFERENCES Question(question_id),  
    INDEX (question_id)  
);
```

-- 6. 考试表

```
CREATE TABLE Exam (  
    exam_id INT AUTO_INCREMENT PRIMARY KEY,  
    exam_name VARCHAR(100) NOT NULL,  
    start_time DATETIME NOT NULL,  
    end_time DATETIME NOT NULL,  
    creator_id INT,  
    FOREIGN KEY (creator_id) REFERENCES User(user_id),  
    INDEX (creator_id),  
    CONSTRAINT chk_exam_time CHECK (start_time < end_time)  
);
```

-- 7. 试卷题目关联表

```
CREATE TABLE ExamQuestion (  
    exam_id INT,  
    question_id INT,  
    score INT,  
    PRIMARY KEY (exam_id, question_id),  
    FOREIGN KEY (exam_id) REFERENCES Exam(exam_id),  
    FOREIGN KEY (question_id) REFERENCES Question(question_id),  
    INDEX (question_id),  
    CONSTRAINT chk_eq_score CHECK (score >= 0 AND score <= 100)  
);
```

-- 8. 考试学生关联表

```
CREATE TABLE ExamStudent (  
    exam_id INT,  
    student_id INT,  
    final_score INT DEFAULT 0 NOT NULL,  
    PRIMARY KEY (exam_id, student_id),  
    FOREIGN KEY (exam_id) REFERENCES Exam(exam_id),  
    FOREIGN KEY (student_id) REFERENCES User(user_id),  
    INDEX (student_id),  
    CONSTRAINT chk_es_finalscore CHECK (final_score >= 0 AND final_score <= 100)  
);
```


-- 9. 提交记录表

```
CREATE TABLE Submission (  
    submission_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    question_id INT,  
    exam_id INT NULL,  
    status ENUM('AC','WA','ERROR'),  
    score INT,  
    submit_time DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES User(user_id),  
    FOREIGN KEY (question_id) REFERENCES Question(question_id),  
    FOREIGN KEY (exam_id) REFERENCES Exam(exam_id),  
    INDEX (user_id),  
    INDEX (question_id),  
    INDEX (exam_id),  
    CONSTRAINT chk_sub_score CHECK (score >= 0 AND score <= 100)  
);
```

感谢聆听

Q & A

欢迎提问 · 交流探讨

